lRepublic of the Philippines

Laguna State Polytechnic University

Province of Laguna

COLLEGE OF COMPUTER STUDIES

C# Console Application Final Project Proposal

Proponent/Researcher:	1. Piñana, Alexander Jr. G. 2. Cabangal, Shelldon Reygie B.
Project Title: (5 to 15 word count): Development of an Interactive C# Console-Based Tic-Tac-Toe Game Application with Score Tracking	
Introduction	Traditional board games, such as Tic-Tac-Toe, have long been a popular pastime for casual recreation and cognitive engagement. However, relying entirely on physical materials like paper and pencils or separate game boards often presents minor inconveniences, such as the need for physical supplies, manual setup, and the potential for human error in tracking turns, validating moves, or determining final win and draw conditions. In casual settings, keeping track of game states manually can occasionally lead to disputes or oversights. To address these limitations, this project introduces an interactive C# Console-Based Tic-Tac-Toe Application. The proposed software system digitizes the classic two-player board game into a lightweight, accessible command-line interface. Instead of manual record-keeping, the application assists users through basic data input (allowing players to input their names and select board positions using single-digit keyboard entries), automated processing (evaluating game rules, tracking move counts, and verifying winning combinations across rows, columns, and diagonals), and dynamic report display (rendering an updated visual 3x3 grid directly in the console after every turn). The relevance of this project lies in its ability to provide instant, error-free gameplay while serving as a practical, foundational application of core programming concepts such as conditional logic, loops, and state management. The potential benefits include offering reliable, straightforward entertainment for users of all ages, while also demonstrating how basic computational logic can streamline and automate traditional manual tasks.
Statement of the Problem	Statement of the Problem Main Problem: Traditional physical methods of playing Tic-Tac-Toe using paper and pencils lack automation, making them prone to human error in tracking turns, validating moves, and determining win or draw conditions. Furthermore, manual score and game state tracking can lead to confusion or disputes during casual play. Specific Questions: 1) How can the application successfully accept, store, and process player names and numeric move inputs (1–9)? 2) How can the system validate player entries to prevent illegal moves, such as choosing already occupied slots or typing invalid characters?

	3) How can the program automatically evaluate and detect winning combinations (rows, columns, diagonals) or a draw after every turn? 4) How can the system manage post-game operations, allowing players to seamlessly choose between restarting the game or terminating the application?
Objective of the Study	Objective of the Study General Objective: ● To develop a functional, interactive C# console-based Tic-Tac-Toe game application that automates two-player gameplay, move validation, win-loss detection, and session management using basic programming constructs. Specific Objectives: ● To develop a simple C# console application that addresses the limitations of manual board game tracking. ● To accept and process user inputs (player names and slot choices) using variables and appropriate data types (string, char, int). ● To implement conditional statements (if-else) and looping structures (while) to control game flow and alternate turns between players. ● To perform necessary logical evaluations, such as checking winning patterns across rows, columns, and diagonals, and tracking move counts. ● To display accurate and organized results through the console interface, including the real-time visual rendering of the 3x3 game board. ● To test the program and verify that its features—including input validation and restart/exit choices—work reliably without crashing.
Scope and Limitation of the Study:	Scope: ● The project is a local, two-player console application written in C#. ● Features an interactive 3x3 grid using numeric keys (1 through 9) via single-keystroke inputs (Console.ReadKey). ● Includes player name registration, real-time score tracking across multiple rounds (scoreA and scoreB), automated turn alternation, move validation, win/draw detection, and an interactive post-game menu ([1] End | [2] Restart). ● Strictly utilizes basic programming constructs: variables, console input/output (Console.ReadLine, Console.ReadKey, Console.Clear), conditional statements (if-else), and looping statements (while). Limitations: ● The system does not feature a Graphical User Interface (GUI), web integration, mobile deployment, or online multiplayer capabilities. ● It does not include an AI or computer opponent; it strictly requires two human players on the same machine. ● It does not involve complex databases or file handling, as game data resets upon exiting.

Review Literatures and System	● Existing Approaches: Traditional Tic-Tac-Toe is commonly played using physical pen and paper, which requires manual grid drawing and constant human vigilance to track marks (X and O). Digital counterparts on modern web browsers or mobile platforms often rely on complex graphical frameworks or cloud-based networking for multiplayer features. ● Limitations of Existing Practices: Physical board setups lack automated rule enforcement, leaving them vulnerable to human errors or disputes. On the other hand, commercial digital applications often introduce unnecessary technical complexity, making them unsuitable for beginner programming students learning foundational logic. ● How the Proposed Application Improves: This C# console application bridges the gap by offering a lightweight, digitized version of the game that automates board rendering, move checking, and win detection. It delivers an interactive user experience while strictly adhering to basic programming logic. ● Credible Context: Developing command-line applications reinforces logical reasoning, algorithmic problem-solving, and syntax mastery among introductory programming students (Deitel & Deitel, 2017).
References:	Deitel, P., & Deitel, H. (2017). C# 6 for programmers. Pearson Education. Microsoft. (2025). C# guide - Tour of C#. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/

Prepared by:

SHELLDON REYGIE B. CABANGAL
ALEXANDER JR G PIÑANA

Business and Functional Requirements

Business Requirement ID	**Business Requirement Description**	**Functional Requirement ID**	**Functional Requirement Description**
BR-001	Simplify the setup and management of a multi-round two-player game session.	FR-001	The system shall allow the user to input names for Player 1 and Player 2.
		FR-002	The system shall display an interactive 3x3 grid and track running scores for both players across matches.
BR-002	Ensure accurate turn progression and responsive single key move validation.	FR-003	The system shall capture single-keystroke inputs (1-9) without requiring the Enter key to alternate turns between ‘X’ and ‘O’.
		FR-004	The system shall validate each move to ensure the chosen slot is valid and has not been previously occupied.

	.		
BR-003	Automate game outcome evaluation and session control.	FR-005	The system shall check all winning patterns (rows, columns, diagonals) and detect draw conditions upon 9 moves.
		FR-006	The system shall provide a single-key post-game menu allowing players to restart the match or view final scores before exiting.